

Learn This Codebase: From Zero to Professional

A Complete Beginner's Guide to the AME Pro AI Workstation

Welcome! This guide will teach you how this entire project works, starting from absolute zero. No previous coding experience needed. We'll go folder by folder, file by file, concept by concept.

TABLE OF CONTENTS

1. [What Is This Project?](#)
 2. [What Is Programming?](#)
 3. [The Technologies Used \(Your Toolbox\)](#)
 4. [The Folder Structure \(Your Filing Cabinet\)](#)
 5. [The Root Files \(The Foundation\)](#)
 6. The `src/` Folder (What Users See)
 7. The `electron/` Folder (The Engine Room)
 8. The `prompts/` Folder (AI Instructions)
 9. [How It All Connects Together](#)
 10. [Key Programming Concepts Explained](#)
 11. [Glossary](#)
-

1. What Is This Project?

AME Pro AI Workstation is a desktop application (a program you install on your computer, like Microsoft Word or Excel) designed for South African accounting, tax, audit, and finance professionals.

Think of it like having a smart assistant on your computer that:

- Lets you chat with AI (like ChatGPT or Claude) about accounting questions
- Helps with SA tax, auditing, bookkeeping, payroll, and quantity surveying
- Stores your conversations and client information in a local database
- Can generate documents (Word, Excel, PDF)
- Works offline on your computer (not just in a web browser)

2. What Is Programming?

Programming is writing **instructions** for a computer to follow. Just like a recipe tells a chef what to do step by step, code tells a computer what to do step by step.

Example from real life:

```
Recipe: Make Tea
1. Boil water
2. Put teabag in cup
3. Pour water into cup
4. Wait 3 minutes
5. Remove teabag
6. Add sugar if wanted
```

The same idea in code (JavaScript):

```
function makeTea(wantSugar) {
  boilWater();
  putTeabagInCup();
  pourWater();
  wait(3);
  removeTeabag();
  if (wantSugar) {
    addSugar();
  }
}
```

See? It's the same logic, just written in a language the computer understands.

Key idea: Programming languages

Just like humans speak English, Zulu, or Afrikaans, computers understand programming languages. This project mainly uses:

- **JavaScript** — the main language (files ending in `.js`)
- **JSX** — JavaScript mixed with HTML for building screens (files ending in `.jsx`)
- **CSS** — the language for making things look pretty (colors, sizes, spacing)
- **HTML** — the language for structuring a web page
- **Markdown** — a simple language for writing formatted text (files ending in `.md`, like this file!)

3. The Technologies Used (Your Toolbox)

Think of building this app like building a house. You need different tools:

Technology	What It Is	Real-World Analogy
JavaScript	The main programming language	The language the builders speak
React	A library for building user interfaces (screens)	The interior design system
Electron	Lets you build desktop apps using web technology	The foundation/structure of the house
Vite	A tool that bundles your code and runs it during development	The construction crane
Tailwind CSS	A system for styling (making things look good)	The paint and decoration catalog
SQLite	A small database that stores data in a file	The filing cabinet
Node.js	Lets JavaScript run outside a web browser	The electrical system
npm/bun	Package managers that download code libraries others wrote	The hardware store

What is React?

React is like a set of **building blocks**. Instead of building one giant page, you build small pieces called **components**, then snap them together.

Imagine building with LEGO: - One LEGO block = a Button - Another block = a Chat Message - Another block = a Sidebar - Snap them all together = Your complete app screen

What is Electron?

Normally, JavaScript only runs inside a web browser (like Chrome). **Electron** is a magic wrapper that lets you take a website and turn it into a desktop app that you can install on Windows, Mac, or Linux. Apps like VS Code, Slack, and Discord are built with Electron.

What is a Database?

A database is like a giant, organized spreadsheet. It stores information in **tables** (like Excel sheets), with **rows** (individual records) and **columns** (fields/categories).

This app uses **SQLite** which stores everything in a single file on your computer. No internet needed!

4. The Folder Structure (Your Filing Cabinet)

Here's every folder and what it's for:

```

Accounting-Pro-A.M.E/      <-- The main project folder (the "house")
|
|-- electron/              <-- The "engine room" (backend - behind the scenes)
|   |-- ipc/               <-- Message handlers (phone lines between front and back
|   |-- migrations/        <-- Database update scripts
|   |-- services/          <-- Business logic (the workers doing the real work)
|
|-- prompts/               <-- AI instruction templates
|   |-- accounting/        <-- Bookkeeping prompts
|   |-- auditing/          <-- Audit prompts
|   |-- finance/           <-- Finance prompts
|   |-- hr-payroll/        <-- HR & payroll prompts
|   |-- quantity-surveying/ <-- QS / BOQ prompts
|   |-- sa-tax/            <-- South African tax prompts
|
|-- scripts/               <-- Helper scripts for developers
|
|-- src/                   <-- The "showroom" (frontend - what users see)
|   |-- components/        <-- Reusable building blocks (LEGO pieces)
|   |   |-- ai/            <-- Chat-related components
|   |   |-- clients/       <-- Client management components
|   |   |-- home/          <-- Home page components
|   |   |-- layout/        <-- Page structure components
|   |   |-- shared/        <-- Components used everywhere
|   |   |-- sidebar/       <-- Sidebar components
|   |-- contexts/          <-- Shared data (global state)
|   |-- hooks/             <-- Reusable logic
|   |-- pages/             <-- Full pages/screens
|   |-- utils/             <-- Helper functions (tools)
|
|-- Various config files    <-- Setup and settings files

```

Think of it this way: - `src/` = The showroom (what customers see in a shop)
 - `electron/` = The back office and warehouse (where the real work happens) -
`prompts/` = The instruction manuals for the AI - Config files = The building permits and blueprints

5. The Root Files (The Foundation)

These files sit at the top level of the project. They configure how everything works.

`package.json` — The Project's ID Card

```
{
  "name": "ame-pro-workstation",
  "version": "1.0.0",
  "description": "AME Pro AI Workstation – Desktop App for SA Professionals"
}
```

What it does: This is like the birth certificate of the project. It tells: - **name:** The project's name (`ame-pro-workstation`) - **version:** What version it is (`1.0.0` = first major release) - **description:** What the project does - **scripts:** Commands you can run (like `npm run dev` to start developing) - **dependencies:** Other people's code this project needs to work (like needing flour and eggs to bake a cake) - **devDependencies:** Tools only needed during development, not in the final app

Key dependencies explained: | Package | What It Does | |---|---| | `react` | Builds the user interface (screens) | | `react-dom` | Connects React to the web page | | `react-router-dom` | Lets you navigate between pages (like clicking links) | | `better-sqlite3` | The database (stores data on your computer) | | `exceljs` | Creates Excel spreadsheets | | `docx` | Creates Word documents | | `pdf-lib` | Creates PDF documents | | `recharts` | Draws charts and graphs | | `lucide-react` | Provides icons (small pictures like a house icon, settings gear, etc.) | | `tailwind-merge` | Helps combine CSS styling classes | | `date-fns` | Makes working with dates easier | | `uuid` | Generates unique IDs (like ID numbers for data) |

`index.html` — The Front Door

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>AME Pro AI Workstation</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Line by line: - `<!DOCTYPE html>` — Tells the browser "this is an HTML page" - `<html lang="en">` — The page is in English - `<head>` — Information about the

page (not visible to users) - `<meta charset="UTF-8">` — Use standard text encoding (supports all characters) - `<title>` — The text shown in the window's title bar - `<body>` — The visible part of the page - `<div id="root"></div>` — An empty box. React will fill this box with the entire app. Think of it as an empty picture frame that React fills with a painting. - `<script src="/src/main.jsx">` — "Hey browser, load and run this file to start the app"

`vite.config.js` — The Build Tool Settings

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  build: {
    chunkSizeWarningLimit: 550,
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ['react', 'react-dom'],
          charts: ['recharts'],
          icons: ['lucide-react'],
        },
      },
    },
  },
})
```

What it does: Configures Vite (the build tool). Think of Vite as a chef that takes all your raw ingredients (code files) and cooks them into a finished meal (the final app).

- `plugins: [react()]` — "Use the React recipe"
- `manualChunks` — "Split the final code into separate packages so it loads faster" (like dividing a big delivery into smaller boxes)

tailwind.config.js — Styling Settings

```
export default {
  content: [
    "./index.html",
    "./src/**/*.{js,ts,jsx,tsx}",
  ],
  theme: { extend: {} },
  plugins: [],
}
```

What it does: Tells Tailwind CSS which files to scan for styling classes. - `"./src/**/*.{js,ts,jsx,tsx}"` — "Look in every file inside `src/` and all its subfolders" (the `**` means "any depth of folders", `*` means "any filename")

postcss.config.js — CSS Processing Pipeline

```
export default {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
}
```

What it does: Runs CSS through two processors: 1. **tailwindcss** — Converts Tailwind class names into actual CSS 2. **autoprefixer** — Adds browser-specific prefixes so styles work in all browsers

electron-builder.yml — Packaging Settings

This tells the system how to package the app for different operating systems (Windows `.exe`, Mac `.dmg`, Linux `.AppImage`). Like instructions for putting the finished product in a box for shipping.

vercel.json — Web Deployment Settings

```
{
  "framework": "vite",
  "installCommand": "npm ci",
  "buildCommand": "npx vite build",
  "outputDirectory": "dist"
}
```


What it does: If you wanted to deploy a web version, this tells Vercel (a hosting service) how to build it.

6. The `src/` Folder (What Users See)

This is the **frontend** — everything the user sees and interacts with.

`src/main.jsx` — The App's Ignition Key

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { HashRouter } from 'react-router-dom';
import { ThemeProvider } from './contexts/ThemeContext';
import { AIProviderProvider } from './contexts/AIProviderContext';
import { WorkspaceProvider } from './contexts/WorkspaceContext';
import { ToastProvider } from './components/shared/Toast';
import App from './App.jsx';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <HashRouter>
      <ThemeProvider>
        <AIProviderProvider>
          <WorkspaceProvider>
            <ToastProvider>
              <App />
            </ToastProvider>
          </WorkspaceProvider>
        </AIProviderProvider>
      </ThemeProvider>
    </HashRouter>
  </React.StrictMode>
);
```

Line by line in plain language:

1. **Lines 1-8** (`import ...`): "Go fetch these tools and building blocks I need."
This is like a chef gathering ingredients before cooking. Each `import` brings in code from another file.
2. **Line 10** (`import './index.css'`): "Load the stylesheet" (the visual design rules)
3. **Line 12** (`ReactDOM.createRoot(document.getElementById('root'))`): "Find that empty `<div id='root'>` box in index.html and get ready to fill it"

4. `.render(...)` with the nested providers: "Fill the box with the app."

The nesting looks confusing, but think of it like **Russian nesting dolls** (Matryoshka dolls): `React.StrictMode` (outermost doll – extra safety checks during development) `HashRouter` (handles page navigation – "which page am I on?") `ThemeProvider` (provides dark/light mode to everyone inside) `AuthProviderProvider` (provides AI settings to everyone inside) `WorkspaceProvider` (provides conversation data to everyone inside) `ToastProvider` (provides notification popups to everyone inside) `App` (the actual app – innermost doll)

Each "Provider" is like a **radio station**. Any component inside it can "tune in" and receive the data it broadcasts. This is called **Context** in React.

`src/App.jsx` — The Traffic Director

```
import { Routes, Route } from 'react-router-dom';
import MainLayout from './components/layout/MainLayout';
import Dashboard from './pages/Dashboard';
import Workspace from './pages/Workspace';
import Settings from './pages/Settings';

export default function App() {
  return (
    <Routes>
      <Route element={<MainLayout />}>
        <Route index element={<Dashboard />} />
        <Route path="workspace" element={<Workspace />} />
        <Route path="workspace/:domain" element={<Workspace />} />
        <Route path="workspace/:domain/:conversationId" element={<Workspace />} />
        <Route path="settings" element={<Settings />} />
      </Route>
    </Routes>
  );
}
```

In plain language: This is like a receptionist at a building: - Someone arrives at the front door (`/`) → Send them to the **Dashboard** - Someone asks for "workspace" (`/workspace`) → Send them to the **Workspace** - Someone asks for "workspace/sa-tax" → Send them to the Workspace for the **SA Tax** domain - Someone asks for "settings" → Send them to **Settings**

The `MainLayout` wraps everything, meaning every page gets the same sidebar, header, and title bar. Think of it as the building itself — the routes just determine which room you enter.

The `:domain` and `:conversationId` parts with colons are **variables** — they can be anything. Like a hotel room number: the hotel structure is the same, but the room number changes.

`src/index.css` — The Global Stylesheet

```
@tailwind base;
@tailwind components;
@tailwind utilities;

@media print { ... }
```

- The three `@tailwind` lines load Tailwind's styling system
- The `@media print` section defines how the page looks when printed (for invoices)

`src/pages/` — The Full Screens

These are the main pages of the app, like rooms in a building.

`src/pages/Dashboard.jsx` — The Home Screen

This is what you see when you first open the app. It shows: - A greeting ("Good morning!" based on time of day) - **Domain cards** — clickable tiles for each professional area (Tax, Audit, Finance, HR/Payroll, etc.) - **Recent conversations** — your latest chats

Key concept — `useState` :

```
const [greeting, setGreeting] = useState('Good evening');
```

This creates a **variable that React watches**. When it changes, the screen updates automatically. Think of it like a scoreboard at a sports game — when the score changes, the display updates. - `greeting` = the current value - `setGreeting` = the function to change the value - `'Good evening'` = the starting value

Key concept — `useEffect` :

```
useEffect(() => {
  const hour = new Date().getHours();
  if (hour < 12) setGreeting('Good morning');
  else if (hour < 17) setGreeting('Good afternoon');
  else setGreeting('Good evening');
}, []);
```

This runs code **when the component first appears on screen**. The `[]` at the end means "run this once." It checks the current hour and sets the greeting accordingly.

`src/pages/Workspace.jsx` — The Chat Page

This is where you actually talk to the AI. It: - Reads the URL to know which domain and conversation you're in - Creates a new conversation if needed - Shows the `ChatInterface` component (the actual chat)

`src/pages/Settings.jsx` — The Settings Page

Where you configure the app: - **API Keys** — passwords to connect to AI services (Claude, OpenAI, DeepSeek, Kimi) - **Theme** — switch between dark and light mode - **Backup** — save and restore your database - **Usage stats** — how many messages you've sent, tokens used, cost in ZAR

`src/components/` — The Building Blocks

Components are **reusable pieces** of the screen. Like how every room in a hotel has the same type of door handle, light switch, and curtain rail.

`src/components/layout/` — Page Structure

File	What It Does
<code>MainLayout.jsx</code>	The page skeleton — TitleBar on top, Sidebar on left, content in the middle
<code>TitleBar.jsx</code>	The very top bar with minimize/maximize/close buttons (like any Windows app)
<code>Sidebar.jsx</code>	The left panel with conversation list, search, domain filters
<code>Header.jsx</code>	The bar at the top of the content area showing current location

`MainLayout.jsx` in plain language:

```
+-----+
| TitleBar (minimize, maximize, close buttons) |
+-----+-----+
|          | Header (shows where you are)      |
| Sidebar  |-----+
| (list of |                                     |
| chats,   | Main Content Area                 |
| search,  | (Dashboard / Workspace / Settings) |
| filters) |                                     |
|          |                                     |
+-----+-----+
```

`src/components/ai/` — Chat Components

These are the pieces that make up the chat experience:

File	What It Does	Real-World Analogy
<code>ChatInterface.jsx</code>	The main chat screen with message list and input box	The WhatsApp/iMessage screen
<code>MessageBubble.jsx</code>	A single message (user or AI)	One speech bubble in a chat
<code>MarkdownRenderer.jsx</code>	Renders formatted text from AI (bold, code, lists)	A text formatter
<code>ChatToolbar.jsx</code>	Toolbar buttons above the input box	The formatting bar in Word
<code>DocumentToolbar.jsx</code>	Buttons for generating documents	Export/print buttons
<code>DocumentActions.jsx</code>	Actions for document generation (Word, Excel, PDF)	"Save As" menu
<code>FileUploadButton.jsx</code>	Button to upload files for AI to read	The paperclip in email
<code>ProviderSelector.jsx</code>	Dropdown to pick which AI model to use	Switching between radio stations
<code>QuickStartPrompts.jsx</code>	Suggested prompts to get started	"Frequently asked questions"
<code>MemoryPanel.jsx</code>	Shows what the AI remembers about you	The AI's notebook
<code>SkillManager.jsx</code>	Manages AI skills and capabilities	The AI's skill list
<code>WorkflowIndicator.jsx</code>	Shows when AI is doing a multi-step task	A progress bar

`ChatInterface.jsx` deep dive:

This is the heart of the app. Here's what it does, step by step:

1. **Receives messages** from the conversation context
2. **Shows a text input box** at the bottom for you to type
3. **When you press Send:**
4. Takes your message
5. Adds it to the chat as a "user" message
6. Sends it to the AI service through the Electron backend

7. Receives the AI's response
8. Adds the response to the chat as an "assistant" message
9. **Handles file uploads** — you can attach files for the AI to analyze
10. **Auto-scrolls** to the latest message

`MessageBubble.jsx` **deep dive:**

Each chat message is displayed as a "bubble," like in WhatsApp: - **User messages** appear on one side with one color - **AI messages** appear on the other side with another color - Shows the AI model name, token count, and cost in ZAR - Has a "Copy" button and a "Retry" button

`src/components/home/DomainCard.jsx` — **Service Area Card**

A clickable card showing an icon, name, and description for each professional domain (like "SA Tax," "Audit," "Payroll"). Used on the Dashboard.

`src/components/clients/ClientManager.jsx` — **Client Panel**

A panel for managing client information — names, contacts, projects.

`src/components/shared/Toast.jsx` — **Notification Popups**

Those small notifications that pop up briefly (like "Message sent!" or "Error!"). Called "toast" because they pop up like toast from a toaster.

`src/components/sidebar/` — **Sidebar Parts**

File	What It Does
<code>ConversationList.jsx</code>	The list of all your conversations, grouped by date
<code>ConversationItem.jsx</code>	A single conversation entry (title, date, domain badge)

`src/contexts/` — **Shared Data (Global State)**

Contexts are like **bulletin boards** that everyone can read. Instead of passing information from parent to child to grandchild (like a game of telephone), contexts let any component access shared data directly.

src/contexts/ThemeContext.jsx — Dark/Light Mode

```
const ThemeContext = createContext();

export function ThemeProvider({ children }) {
  const [isDark, setIsDark] = useState(true);
  // ...
  return (
    <ThemeContext.Provider value={{ isDark, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}
```

In plain language: - Creates a "bulletin board" called `ThemeContext` - Puts `isDark` (true/false) and `toggleTheme` (a function to switch) on the bulletin board - Any component anywhere in the app can read this bulletin board to know if it should show dark or light colors

src/contexts/AIProviderContext.jsx — AI Settings

Manages: - Which AI providers are available (Claude, OpenAI, DeepSeek, Kimi) - Which model is currently selected - Whether each provider is working (health checks) - API key management

Contains a **model catalog** — a list of all available AI models with their names, IDs, and capabilities.

src/contexts/WorkspaceContext.jsx — Conversation Data

Manages everything about your work: - All conversations (create, read, update, delete) - The currently active conversation - Domain filtering (show only Tax conversations, only Audit, etc.) - Search functionality - File attachments - Grouping conversations by date ("Today," "Yesterday," "Last Week," etc.)

src/hooks/ — Reusable Logic

Hooks are **reusable behaviors** you can plug into any component. Think of them like power tools — instead of building the same logic every time, you just plug in the hook.

`src/hooks/useAIChat.js` — Chat Logic

Handles the back-and-forth of chatting with AI: 1. Take the user's message 2. Send it to the backend 3. Wait for the AI response 4. Return the response 5. Handle errors

`src/hooks/useProviderHealth.js` — Connection Checker

Periodically checks if the AI services are online and responding, like pinging a website to see if it's up.

`src/utils/` — Helper Functions

Small, focused tools that do one thing well.

`src/utils/formatCurrency.js` — Money Formatting

```
// Turns 1234.56 into "R 1 234,56"  
export function formatZAR(amount) { ... }
```

Formats numbers as South African Rand (ZAR) using the proper format.

`src/utils/dateHelpers.js` — Date Formatting

```
// Turns a date into "12 March 2026" (SA format)  
export function formatDateSA(date) { ... }  
  
// Calculates the SA tax year (March 1 to February 28/29)  
export function getTaxYear(date) { ... }
```

7. The `electron/` Folder (The Engine Room)

This is the **backend** — the behind-the-scenes machinery. Users never see this directly, but it does all the heavy lifting.

`electron/main.js` — The Engine Starter

This is like turning the key to start a car engine. It:

1. **Imports all services** (database, AI, files, etc.)

2. **Imports all IPC handlers** (the phone lines)
3. **Creates the app window** with specific size (1400x900), dark background, and security settings
4. **Initializes the database** when the app starts
5. **Registers all handlers** so the frontend can communicate with the backend
6. **Handles window controls** (minimize, maximize, close)

```
app.whenReady().then(async () => {  
  await database.initialize();      // Set up the database  
  memoryService.init(database);     // Set up AI memory  
  skillService.init(database);      // Set up AI skills  
  backupService.runOnStartup();     // Check for backups  
  registerAllHandlers();           // Set up all communication  
  createWindow();                  // Show the window  
});
```

In plain language: "When the app is ready to go: set up the database, set up memory, set up skills, check backups, set up all communication channels, then show the window to the user."

`electron/preload.js` — The Translator

This is the **bridge** between the frontend (what users see) and the backend (the engine room). For security, Electron doesn't let the frontend talk directly to the backend. The preload file creates a safe, controlled set of functions the frontend is allowed to use.

Think of it like a reception desk at a hospital: - You (the frontend) can't walk into the operating room directly - You tell the receptionist (preload) what you need - The receptionist passes the message to the doctor (backend) - The doctor sends the answer back through the receptionist

It exposes 60+ functions organized by category: - `window.api.sendToAI()` — Send a message to AI - `window.api.dbGetConversations()` — Get all conversations - `window.api.saveDocument()` — Save a document - `window.api.executeCommand()` — Run a terminal command - And many more...

`electron/services/` — The Workers

Each service is like a specialized worker that handles one area:

database.js — The Filing Clerk

Sets up and manages the SQLite database. Creates these tables (like Excel spreadsheets):

Table	What It Stores
users	User accounts
clients	Client information (name, company, contact, tax numbers)
projects	Projects linked to clients
conversations	Chat conversations (title, domain, dates)
messages	Individual messages within conversations
documents	Generated documents
prompts	AI prompt templates
audit_trail	Log of everything that happened (for security)
usage_log	AI usage tracking (tokens, cost)
ai_memory	What the AI remembers about you
skills	AI skills and capabilities

ai-gateway.js — The AI Switchboard

This is the **central hub** for talking to AI services. Like a telephone switchboard operator who connects your call to the right person:

1. Receives a request ("Send this message to Claude")
2. Formats it correctly for that specific AI service
3. Sends it over the internet
4. Receives the response
5. Formats it back into a standard format
6. Tracks usage and calculates cost in ZAR

Cost tracking example:

```
const COST_PER_MILLION_TOKENS_ZAR = {
  'claude-opus-4-6':      { input: 275.00, output: 1375.00 },
  'claude-sonnet-4-6':   { input: 55.00,  output: 275.00 },
  'gpt-4.1':             { input: 36.67,  output: 146.67 },
  // ... more models
};
```

This means: for every million "tokens" (roughly 750,000 words) sent to Claude Opus, it costs R275 for input and R1,375 for output.

`ai-providers.js` — The AI Phone Book

Contains the configuration for each AI provider: - **Claude** (Anthropic) — endpoint URL, available models, how to format requests - **OpenAI** (GPT) — same structure - **DeepSeek** — same structure - **Kimi** (Moonshot) — same structure

Think of it as an address book: "To call Claude, dial this number, speak in this format."

`document-engine.js` — The Document Factory

Creates professional documents: - Word documents (`.docx`) - Excel spreadsheets (`.xlsx`) - PDF files (`.pdf`)

`file-service.js` — The File Handler

Reads and writes files on your computer. Can read uploaded documents (PDF, Word, Excel) and extract their text content for the AI to analyze.

`backup-service.js` — The Safety Net

Automatically backs up your database so you don't lose data. Like having a photocopy of all your important documents.

`keychain.js` — The Safe

Securely stores your API keys (the passwords for AI services). Uses the operating system's built-in secure storage (like Keychain on Mac or Credential Manager on Windows).

`memory-service.js` — The AI's Notebook

Lets the AI remember things about you across conversations. Like a personal assistant who writes down your preferences.

`prompt-engine.js` — The Script Writer

Loads the prompt templates from the `prompts/` folder and combines them with your message to create the best possible instruction for the AI.

`terminal-service.js` — The Command Runner

Lets the AI run commands on your computer (like a programmer would in a terminal). Used for things like running code or checking system info.

`web-search-service.js` — The Internet Researcher

Lets the AI search the web for current information.

`workflow-engine.js` — The Project Manager

Manages multi-step tasks. When the AI needs to do several things in order (like "research, then write, then review"), this service coordinates the steps.

`skill-service.js` — The Skill Trainer

Manages custom skills — specific capabilities the AI can learn and use.

`electron/ipc/` — The Phone Lines

IPC stands for **Inter-Process Communication**. Remember the hospital analogy? These files are the actual message routes between the receptionist and the doctors.

Each handler file matches a service:

Handler File	Connects Frontend To
<code>ai-handlers.js</code>	AI Gateway (send messages, check health)
<code>db-handlers.js</code>	Database (get/save conversations, messages)
<code>doc-handlers.js</code>	Document Engine (generate documents)
<code>file-handlers.js</code>	File Service (read/write files)
<code>settings-handlers.js</code>	Keychain & Settings (API keys, preferences)
<code>terminal-handlers.js</code>	Terminal Service (run commands)
<code>memory-handlers.js</code>	Memory Service (AI memory)
<code>search-handlers.js</code>	Web Search Service
<code>skill-handlers.js</code>	Skill Service
<code>client-handlers.js</code>	Client management

How a message flows:

You type "What is VAT?" and press Send

1. ChatInterface.jsx calls `window.api.sendToAI(message)`
2. `preload.js` converts this to an IPC call
3. `ai-handlers.js` receives the IPC call
4. `ai-handlers.js` calls `aiGateway.sendRequest()`
5. `ai-gateway.js` formats the request for Claude/OpenAI
6. `ai-gateway.js` sends it over the internet
7. The AI responds
8. The response flows back: `ai-gateway` → `ai-handlers` → `preload` → `ChatInterface`
9. `ChatInterface` shows the response in a `MessageBubble`

`electron/migrations/` — Database Updates

When the database structure needs to change (like adding a new column to a table), migration files handle the update without losing existing data. Like renovating a room while keeping all the furniture.

8. The `prompts/` Folder (AI Instructions)

These are **Markdown files** (`.md`) that contain expert knowledge for each professional domain. They're like cheat sheets that the AI reads before answering your questions.

File	What It Contains
<code>accounting/bookkeeping.md</code>	South African bookkeeping rules and standards
<code>auditing/external-audit.md</code>	External audit procedures and standards
<code>finance/general.md</code>	General finance knowledge
<code>hr-payroll/payroll.md</code>	South African payroll rules, UIF, PAYE
<code>quantity-surveying/boq.md</code>	Bill of Quantities and QS practices
<code>sa-tax/individual-tax.md</code>	Individual income tax rules for SA
<code>sa-tax/vat.md</code>	VAT rules for South Africa

When you start a chat in the "SA Tax" domain, the app loads `sa-tax/individual-tax.md` and sends it to the AI along with your question. This makes the AI much more accurate because it has the specific SA rules right in front of it.

9. How It All Connects Together

Here's the complete flow of the application:

Starting the App

1. You double-click the app icon
2. Electron starts up (`electron/main.js`)
3. Database is initialized (tables created if first time)
4. Services are set up (memory, skills, backup)
5. Communication channels (IPC) are registered
6. The window appears showing `index.html`
7. `index.html` loads `src/main.jsx`
8. `main.jsx` sets up React with all the providers
9. `App.jsx` reads the URL and shows the Dashboard
10. Dashboard shows domain cards and recent conversations

Sending a Chat Message

1. You click a domain card (e.g., "SA Tax")
2. A new conversation is created in the database
3. The Workspace page loads with ChatInterface
4. The relevant prompt template is loaded (sa-tax/individual-tax.md)
5. You type "How do I calculate provisional tax?" and press Enter
6. Your message is saved to the database
7. The message + prompt template is sent to the AI (via gateway)
8. The AI processes your question with SA tax knowledge
9. The AI's response comes back
10. The response is saved to the database
11. The response appears in a MessageBubble on your screen
12. Token count and cost (in ZAR) are displayed

Architecture Diagram



10. Key Programming Concepts Explained

Variables — Named Containers

```
const name = "AME Pro";           // A text value (called a "string")
const version = 1.0;              // A number
const isReady = true;             // A true/false value (called a "boolean")
const users = ["Alice", "Bob"];  // A list (called an "array")
```

Think of variables like labeled jars in a kitchen. The label is the name, the contents is the value.

- `const` = "This value won't change" (constant)
- `let` = "This value might change later"

Functions — Reusable Actions

```
function greet(name) {
  return `Hello, ${name}!`;
}

greet("Thabo"); // Returns: "Hello, Thabo!"
greet("Nomsa"); // Returns: "Hello, Nomsa!"
```

A function is like a **vending machine**: you put something in (the name), it does something, and gives you something back (the greeting).

Arrow Functions — Shorthand Functions

```
// Regular function
function add(a, b) {
  return a + b;
}

// Arrow function (same thing, shorter syntax)
const add = (a, b) => a + b;
```

Arrow functions (`=>`) are just a shorter way to write functions. You'll see them everywhere in this project.

Objects — Grouped Information

```
const client = {  
  name: "Thabo Mokoena",  
  company: "TM Consulting",  
  taxNumber: "1234567890",  
  isActive: true,  
};
```

An object groups related information together, like a contact card.

Arrays — Lists of Things

```
const domains = ["SA Tax", "Audit", "Finance", "Payroll"];  
domains[0]; // "SA Tax" (counting starts at 0, not 1!)  
domains[3]; // "Payroll"
```

`import` / `export` — Sharing Between Files

```
// In formatCurrency.js (SHARING something)  
export function formatZAR(amount) { ... }  
  
// In Settings.jsx (USING the shared thing)  
import { formatZAR } from '../utils/formatCurrency';
```

`export` = "I'm making this available for others to use" `import` = "I want to use something from another file"

JSX — HTML Inside JavaScript

```
function WelcomeMessage({ name }) {  
  return (  
    <div className="p-4 bg-blue-500">  
      <h1>Welcome, {name}!</h1>  
    </div>  
  );  
}
```

JSX lets you write HTML-like code inside JavaScript. The curly braces `{name}` mean "insert the value of this variable here." It's like a mail merge in Word where `{name}` gets replaced with the actual name.

Props — Passing Data to Components

```
// Parent passes data DOWN to child
<DomainCard name="SA Tax" description="Individual & corporate tax" icon={Calculator} />

// Child RECEIVES the data
function DomainCard({ name, description, icon }) {
  return <div>{name}: {description}</div>;
}
```

Props are like **function arguments** but for components. The parent says "here's your data," and the child uses it.

State (`useState`) — Remembering Things

```
const [count, setCount] = useState(0);
// count = current value (starts at 0)
// setCount = function to change the value

setCount(5); // Now count is 5, and the screen updates!
```

State is data that can **change over time** and **automatically updates the screen** when it does.

Effects (`useEffect`) — Doing Things at the Right Time

```
useEffect(() => {
  // This code runs when the component appears on screen
  fetchConversations();
}, []);

useEffect(() => {
  // This code runs whenever `selectedDomain` changes
  filterConversations(selectedDomain);
}, [selectedDomain]);
```

- `[]` (empty) = "Run once when the component first shows up"
- `[selectedDomain]` = "Run every time selectedDomain changes"

`async` / `await` — Waiting for Slow Things

```
async function sendMessage(text) {  
  const response = await window.api.sendToAI(text);  
  return response;  
}
```

Some operations take time (like sending a message to AI over the internet). `async/await` is like saying "do this, and **wait** until it's done before continuing." Without it, the code would try to use the response before it arrived.

Conditional Rendering — Show/Hide Based on Conditions

```
{isLoading && <Spinner />}  
{/* If isLoading is true, show the Spinner. Otherwise, show nothing. */}  
  
{error ? <ErrorMessage /> : <SuccessMessage />}  
{/* If there's an error, show ErrorMessage. Otherwise, show SuccessMessage. */}
```

`.map()` — Repeat for Each Item

```
{conversations.map(conv => (  
  <ConversationItem key={conv.id} title={conv.title} />  
))}
```

This takes a list of conversations and creates a `ConversationItem` component for each one. Like a factory assembly line — each item goes through the same process.

Tailwind CSS Classes — Styling Shorthand

Instead of writing CSS in a separate file, Tailwind uses class names directly:

```
<div className="p-4 bg-slate-800 text-white rounded-lg shadow-md">
```

Class	What It Means
<code>p-4</code>	Padding (inner spacing) of size 4
<code>bg-slate-800</code>	Background color: dark slate
<code>text-white</code>	Text color: white
<code>rounded-lg</code>	Rounded corners (large)
<code>shadow-md</code>	Medium drop shadow
<code>flex</code>	Use flexbox layout (items in a row/column)
<code>w-full</code>	Width: 100%
<code>mt-2</code>	Margin-top (outer spacing above) of size 2
<code>hover:bg-blue-600</code>	When mouse hovers: change background to blue

11. Glossary

Term	Meaning
API	Application Programming Interface — a way for programs to talk to each other
API Key	A password/token that identifies you to an external service
Array	A list of items <code>[1, 2, 3]</code>
Async	Code that takes time and doesn't block other code from running
Backend	The behind-the-scenes part users don't see (server, database)
Boolean	A true/false value
Component	A reusable building block of the user interface
Context	A way to share data globally in React without passing through every level
CRUD	Create, Read, Update, Delete — the four basic operations on data
CSS	Cascading Style Sheets — the language for visual styling
Database	Organized storage for data (like a smart spreadsheet)
Dependency	Code written by someone else that your project uses
DOM	Document Object Model — the browser's representation of the page
Electron	Framework for building desktop apps with web technology
Frontend	The part of the app users see and interact with
Function	A reusable block of code that performs a specific task
Git	A system that tracks changes to your code over time
Hook	A React feature that lets you use state and effects in components
HTML	HyperText Markup Language — the structure of web pages
IPC	Inter-Process Communication — how frontend and backend talk
JavaScript (JS)	The programming language of the web
JSON	JavaScript Object Notation — a data format like <code>{"key": "value"}</code>

Term	Meaning
JSX	JavaScript + HTML mixed together (used in React)
Migration	A controlled update to the database structure
Node.js	Lets JavaScript run outside a browser (on servers/desktops)
npm	Node Package Manager — downloads and manages code libraries
Object	A collection of named values <code>{ name: "Thabo", age: 30 }</code>
Package	A bundle of code you can install and use in your project
Props	Data passed from a parent component to a child component
React	A JavaScript library for building user interfaces
Route	A URL path that maps to a specific page/screen
SQLite	A lightweight database stored in a single file
State	Data that changes over time and causes the screen to update
String	A piece of text <code>"Hello"</code>
Tailwind	A CSS framework using utility class names for styling
Token	A unit of text the AI processes (roughly 3/4 of a word)
Variable	A named container that holds a value
Vite	A fast build tool for web projects
ZAR	South African Rand (currency)

Your Learning Path

Now that you understand the big picture, here's what to study next:

1. **HTML basics** — Learn tags like `<div>`, `<h1>`, `<p>`, `<button>`
2. **CSS basics** — Learn colors, spacing, layout (flexbox)
3. **JavaScript basics** — Variables, functions, arrays, objects, if/else
4. **React basics** — Components, props, state, effects
5. **Read this codebase** — Start with simple files like `formatCurrency.js` and `dateHelpers.js`, then work up to `Dashboard.jsx`, then `ChatInterface.jsx`

6. **Try changing things** — Change a color, change the greeting text, add a new domain card. Seeing your changes work is the best way to learn!

This guide was created to help you understand the AME Pro AI Workstation codebase from scratch. Happy learning!